

Entregas da Semana 4: Testes e Qualidade de Software

Entrega 1: Mentalidade de Testes e Caixa-Preta

Fase 1: O Mapa do Tesouro

Objetivo: Entender a diferença entre testar por acaso e projetar testes com método profissional.

- Visão Geral:** Assista aos trechos selecionados de [O Guia Definitivo de Qualidade de Software para Iniciantes](#):
 - `[00:00:25]` Por que a qualidade de software é crucial e os impactos de um bug.
 - `[00:03:45]` O que é testar na prática (Esperado vs. Obtido).
 - `[00:10:54]` Testes manuais vs. Automatizados e o ciclo de reporte.
- Pílula de Arquitetura (10 min):** Assista a [Melhores e piores tipos de teste](#), do trecho `[00:03:27]` até `[00:13:30]`, para entender a pirâmide de testes e como focar em regras de negócio críticas.

Fase 2: Aquecimento Gamificado

Objetivo: Calibrar a mentalidade de “pensar nas bordas” antes de escrever casos formais.

- Obrigatório (mentalidade de QA):** Jogue [The Password Game](#) até a Regra 9. Repare como cada regra nova restringe as entradas válidas e cria comportamentos inesperados no sistema.
- Obrigatório (testes de código):** Acesse o [Code Defenders Puzzles](#).
 - Complete todos os puzzles do nível **beginner**, alternando entre os papéis de **Defender** (escreva um caso de teste que “mate o mutante”: passe no código original, mas falhe no código com defeito injetado) e **Attacker** (crie um mutante que sobreviva aos testes existentes).

Fase 3: O Desafio — Caçador de Bugs na Bilheteria

Objetivo: Projetar casos de teste estruturados usando Particionamento de Equivalência (EP) e Análise de Valor Limite (BVA).

Cenário de Negócio: Você é responsável pelo QA da bilheteria de um museu. Regra de tarifação por visitante:

- Entrada inválida:** idades `< 0` ou `> 120` disparam a exceção `"Idade inválida"`.
- Gratuidade (R\$ 0,00):** de 0 a 5 anos (inclusive).
- Meia-entrada (R\$ 20,00):** de 6 a 17 anos (inclusive) e a partir de 60 anos (inclusive).
- Tarifa cheia (R\$ 40,00):** de 18 a 59 anos (inclusive).

Instruções: Crie o arquivo `MATRIZ_TESTES.md` no seu repositório e complete as colunas **Saída Esperada** e **Técnica** (o CT-01 já está resolvido como exemplo):

ID	Cenário / Descrição	Entrada (idade)	Saída Esperada	Técnica (EP/BVA)
CT-01	Limite inferior inválido	-1	Erro: “Idade inválida”	BVA
CT-02	Borda inferior de gratuidade	0		
CT-03	Borda superior de gratuidade	5		
CT-04	Borda inferior de meia (jovem)	6		
CT-05	Borda superior de meia (jovem)	17		
CT-06	Borda inferior de inteira	18		
CT-07	Borda superior de inteira	59		
CT-08	Borda inferior de meia (idoso)	60		
CT-09	Borda superior válida	120		
CT-10	Limite superior inválido	121		

Critério de Sucesso: `MATRIZ_TESTES.md` salvo na sua pasta do laboratório com todas as saídas e técnicas preenchidas.

🏆 **Conquista — Caçador de Fronteiras:** completou todas as linhas sem consultar gabarito externo.

Entrega 2: Diagnóstico de Código e Bug Report

Fase 1: O que é um Bug Report

Objetivo: Entender por que “não funciona” não é uma descrição de defeito aceitável.

- **Pílula rápida:** um bug report existe para que qualquer engenheiro da equipe consiga reproduzir e isolar a falha em segundos. Passos objetivos valem mais do que descrições longas.
- **Níveis de Severidade:**
 - **Crítica:** interrompe o fluxo principal, crasha o sistema ou causa perda financeira direta.
 - **Alta:** regra de negócio importante é calculada incorretamente em casos de uso frequentes.
 - **Média:** inconsistência restrita a condições de contorno ou cenários pouco comuns.
 - **Baixa:** falha estética, erro ortográfico ou aviso de log que não afeta o resultado final.

Fase 2: Execução e Quebra em Linha de Comando

Objetivo: Colocar a especificação contra a implementação real e ver o teste acusar o defeito.

1. Instale o executor de testes no seu terminal (caso ainda não tenha):

```
pip install pytest
```

2. A equipe de desenvolvimento entregou o arquivo `bilheteria.py`:

```
def calcular_ingresso(idade: int) -> float:
    # Validação de limites gerais
    if idade > 120:
        raise ValueError("Idade inválida")

    if idade <= 5:
        return 0.0
    elif idade < 18:
        return 20.0
    elif idade <= 60:
        return 40.0
    else:
        return 20.0
```

3. Crie o arquivo `test_bilheteria.py` com a suíte de testes de fronteira:

```
from bilheteria import calcular_ingresso
import pytest

def test_gratuidade_bebe():
    assert calcular_ingresso(0) == 0.0
    assert calcular_ingresso(5) == 0.0

def test_meia_jovem():
    assert calcular_ingresso(6) == 20.0
    assert calcular_ingresso(17) == 20.0

def test_inteira():
    assert calcular_ingresso(18) == 40.0
    assert calcular_ingresso(59) == 40.0

def test_meia_idoso():
    assert calcular_ingresso(60) == 20.0
    assert calcular_ingresso(120) == 20.0

def test_idade_negativa():
    with pytest.raises(ValueError):
        calcular_ingresso(-1)

def test_idade_acima_limite():
    with pytest.raises(ValueError):
        calcular_ingresso(121)
```

4. Execute o comando no terminal e analise os relatórios de falha:

```
pytest test_bilheteria.py -v
```

Fase 3: O Desafio — Elaboração do Bug Report

Objetivo: Comunicar uma falha de forma clara, reproduzível e padronizada.

Escolha **um** dos bugs encontrados na execução e preencha o arquivo `BUG_REPORT.md`:

```
**ID do Defeito:** BUG-001
**Título:** [Componente + Condição + Comportamento Incorreto, em uma linha]
**Severidade:** [Crítica | Alta | Média | Baixa]
**Ambiente:** Python 3.x / Função calcular_ingresso
**Passos para Reproduzir:**
1. Importar calcular_ingresso do módulo bilheteria.
2. Chamar a função passando o parâmetro X.
3. Analisar o valor retornado ou exceção lançada.
**Resultado Esperado:** [conforme a especificação]
**Resultado Obtido:** [o retorno incorreto observado]
**Causa Provável e Sugestão de Correção:** [linha do código e ajuste lógico necessário]
```

(Desafio Ninja opcional): corrija `bilheteria.py` até que 100% dos testes passem em verde.

Critério de Sucesso: testes executados localmente, `BUG_REPORT.md` preenchido e commitado na sua pasta no repositório.

🔪 **Conquista — Debugger Novato:** encontrou e documentou 1 defeito.

🔪 **Conquista Bônus — Debugger Mestre:** documentou os 2 defeitos e completou o Desafio Ninja.

Biblioteca de Consulta (Guarde nos Favoritos)

- [Software Testing Fundamentals \(STF\)](#) — guia prático para BVA, EP e níveis de teste.
- [Documentação Oficial do Pytest](#) — guia de comandos, fixtures e assertions.
- [Gherkin Reference Guide \(Cucumber\)](#) — estruturação de cenários em `Dado / Quando / Então`.